

Dabblebase

Ajay Gandecha

Abstract

Graduates of computer science programs are increasingly expected to demonstrate proficiency in full-stack software engineering. Many programs are seeking to incorporate project-based courses where students gain experience designing and building modern full-stack applications. However, traditional full-stack applications require a complex infrastructure setup, including databases, authentication systems, file storage, and realtime pub/sub, which is difficult to manually set up and scale for a large class. Backend-as-a-Service (BaaS) platforms such as Supabase make it easier by providing this infrastructure to students, but are not tailored for classroom use, often providing limited instructor control and access. This paper introduces Dabblebase, a BaaS platform designed specifically for the classroom. Dabblebase enables instructors to provide fully-featured database, authentication, storage, and realtime services for their students' software engineering projects without sacrificing any of the features of mainstream BaaS providers like Supabase. Dabblebase lowers barriers for project-based learning and provides a scalable model for teaching full-stack software engineering using modern and realistic, production-like services in computer science curricula.

Introduction

The technology industry continues to evolve at a rapid rate. As the industry accelerates, requirements for new graduates seeking software engineering roles also has begun to change. Students are often required to discuss or demonstrate proficiency with a variety of software engineering tools and methods during the interview process. Recent surveys suggest that practicing software engineers in industry increasingly feel that new graduates lack experience with realistic tooling, infrastructure, and systems typically used in large scale software projects [1]. In response to this, university programs are seeking ways to design new courses to give students hands-on experience with industry-grade tooling and real-world software engineering practices through developing full-stack software projects. One of the most significant challenges for these classes is the management and scaling of the infrastructure to power these more expansive applications.

At UNC, one of our faculty members developed a new course to bridge this gap - *COMP 423: Foundations of Software Engineering*. Throughout the course, students gain hands-on experience developing at all of the different layers in the traditional three-tier architecture of the full-stack. COMP 423 prioritizes team-based projects to build students' technical communication, collaboration, and project management skills. Students also work on a large full-stack software engineering project developing a feature for a web application used in production department-wide by over one thousand current students. According to survey data taken at the conclusion of the course, COMP 423 received extremely high praise from students. Many students also attributed job offers they received to the experience they gained in COMP 423.

The backend infrastructure for student projects were hosted on Carolina CloudApps, a cloud application platform powered by Red Hat's OpenShift, built on Kubernetes, offered by UNC to students. Students were given access to their own namespaces on CloudApps. Course staff also had access to student namespaces, giving them visibility into students' project configurations for grading purposes. Once given their namespace, students could manually create a database, deploy a containerized app, and connect their app to the created database instance using the generated database's credentials. Every time a student needed to deploy a new assignment, they would need to go through these steps manually.

The setup process for deploying a project in a Kubernetes instance is far from trivial, even for experienced developers. Course staff reported that a significant number of office hours

encounters during final projects resulted in debugging CloudApps setup issues. In addition, some issues at the CloudApps level (such as limits on memory allocation) took time to resolve.

One alternative solution is to use a Backend-as-a-Service (BaaS) platform to provide the infrastructure to power student projects. Supabase is one such provider. Supabase provides students with access to a private PostgreSQL database and authentication services for each of their projects. Supabase even provides S3-like file storage and realtime pub/sub via web sockets, features that are exceptionally difficult to set up manually for students and enable the development of a much larger variety of full-stack applications. Many BaaS providers, including Supabase, offer a generous free tier that comfortably encapsulates typical usage in the academic setting.

As an instructor in the department, I have been working on modernizing our full-stack web development course, *COMP 426: Modern Web Programming*. When designing the new curriculum for COMP 426, my goal was to give students significant hands-on experience developing a large variety of features for production-grade applications to both prepare students for industry and to equip them with the tools to build their own software projects on their own. Rather than having students set up their project infrastructure manually in Carolina CloudApps, COMP 426 opted to use Supabase.

COMP 426 integrated Supabase into its curriculum with largely great success. Two assignments used Supabase extensively. The first, *Oriole*, a Twitter-like social media app, used Supabase's database, authentication, and file storage features. The second, *Alias*, a recreation of the popular social chat-room app Discord, also incorporated Supabase's realtime pub/sub functionality for instant messaging, typing indicators, and user online / offline statuses. Without Supabase, the infrastructure for an assignment with such rich and extensive functionality would have been exceptionally difficult to set up manually. Students also used Supabase to configure the backend infrastructure for their final projects, allowing students to create incredibly feature-rich full-stack applications.

Supabase, as well as many other alternative BaaS providers like Firebase, have limitations that hinder their adoption in computer science classrooms. Supabase pauses projects that have been inactive for a few days, which can interrupt the grading process. In the case projects are paused, COMP 426 course staff must reach out to students and instruct them to unpause their project, which causes significant grading delays. Supabase also has a limit to two active projects per user. In the case that more than two assignments use Supabase, students have to delete an old project or must find clever ways to skirt this limit, which slows down students eager to get started with their assignment. Also, since all Supabase projects are scoped to students' Supabase accounts, course staff do not have visibility into a project's configuration, database, or stored data. If an assignment's grading criteria scrutinizes such configuration, students must export their configuration or submit a screenshot with their submission so the course staff can grade their assignment. Setting up a project with existing data is also a hassle - students must seed their database with test data using the Supabase CLI tool or by importing data with `.csv` files.

This paper introduces Dabblebase, a Backend-as-a-Service platform designed specifically for use in software engineering courses. Dabblebase aims to provide the same powerful features as existing BaaS platforms like Supabase for student projects while improving tooling and setup for instructors and students. This paper also discusses Dabblebase's Client libraries, enabling students to easily connect their apps to Dabblebase's features.

Dabblebase

Design Goals

For Dabblebase to provide value for both students and instructors, its implementation is designed around these core principles:

- **Feature-Rich Tooling:** Tools designed for pedagogical use often feel limited in capability. Dabblebase should offer the same core functionality as other BaaS services (like Supabase) to support a large variety of student projects.

- **Minimal Vendor Lock-In:** If students wish to further develop a project of theirs after a course finishes, they should be able to without having to rewrite large pieces of their app. Dabblebase should be designed in a way that makes it easy to switch to another infrastructure solution.
- **Flexible Support:** Dabblebase should be flexible enough to work with a variety of tech stacks and tools to support the pedagogical goals for a variety of different courses.
- **Features for Instructors:** Dabblebase should make it easy for instructors to create assignments with a customizable configuration to support a wide variety of assignments and courses. It should also be easy for instructional staff to access student projects and their databases for easy grading and debugging during office hours.
- **Ease of Use:** Since the core users of Dabblebase will be students who will often be implementing backend features for the first time, Dabblebase and its APIs must be written in a way that is easy to use, and its documentation must be presented in a way that is digestible and accessible.
- **Pedagogical Value:** Many existing BaaS solutions design their APIs in a way that obscures how features they provide work. Without compromising on usability, Dabblebase's APIs should be designed in a way that reveals to students the core technologies behind how features work (such as cookies / JWTs for authentication, SQL for databases, or web sockets for realtime).
- **Self-Hostable and Open Source:** Dabblebase should be able to be self-hosted by instructors or university departments so their students are able to use it. Dabblebase is a tool for educators and students alike, so it should be entirely open-source for future development.

Core Features

- **Database:** Dabblebase Database provides a private, isolated database for every student project. Dabblebase provides students with database credentials they can use to connect their application backends directly to the database. Instructional staff also have direct access to student databases for grading purposes and for assisting students in office hours, and student databases can be pre-populated with data tailored to specific assignments.
- **Authentication:** Dabblebase Auth manages user authentication for all student projects using UNC authentication, and students are able to configure their projects' backends to verify that signed-in users authenticated through Dabblebase Auth.
- **Realtime:** Dabblebase Realtime exposes web socket endpoints for student projects for realtime updates. These endpoints are used for three specific purposes:
 - **Database Changes:** Student projects can listen for realtime changes to data in their database and respond to those changes in their application's frontend.
 - **Presence:** Students can configure their projects to be able to see who is currently logged in to their apps.
 - **Broadcast:** Student projects can set up channels to send messages back and forth between clients in realtime.

With realtime support, students are able to learn about web sockets and implement them in their projects. This allows for more feature-rich projects. For example, in COMP 426, students use a combination of the realtime features above to recreate Discord, a popular social media and realtime chatting application.

- **Storage:** Dabblebase Storage provides S3-like object storage for student projects, allowing them to work with media such as image uploads in their applications.

Dabblebase's Core Implementation

With all of the features mentioned above implemented, Dabblebase's full architecture is represented by the following diagram:

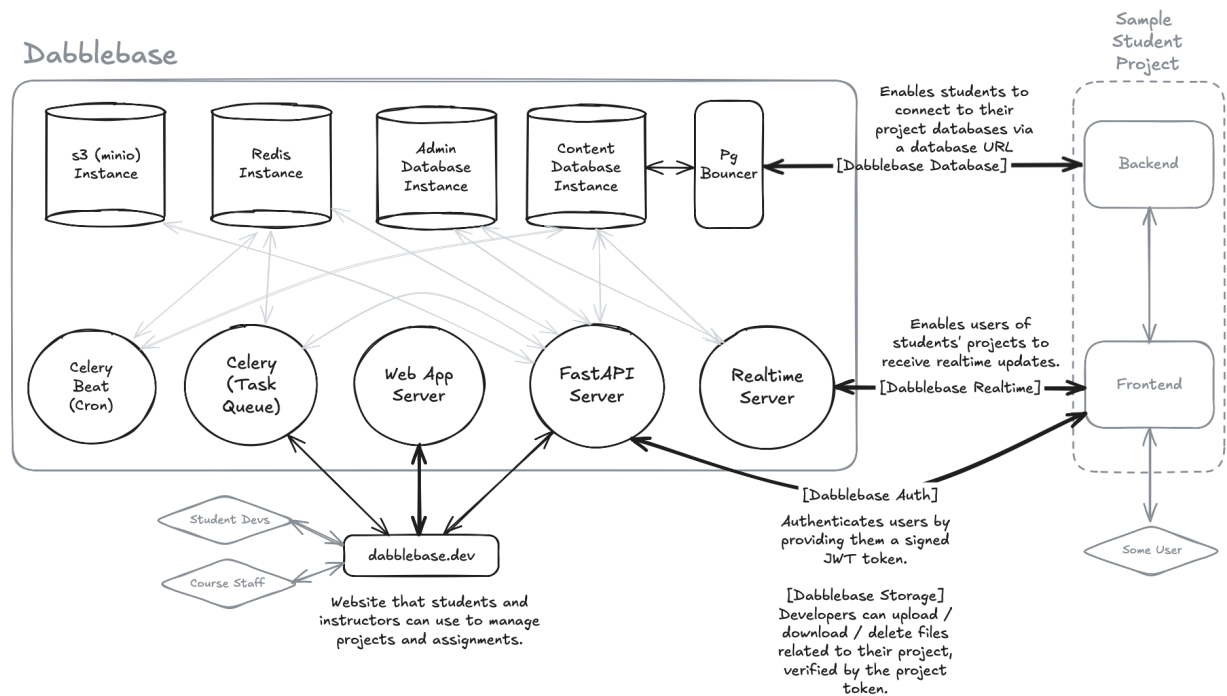


Figure 1: Completed architecture of Dabblebase

In total, Dabblebase encompasses the following components running in parallel that power Dabblebase's core features:

- **Web Server:** Typescript-based server that serves the Dabblebase web application, allowing instructors to manage student projects and students to view credentials to access Dabblebase's services.
- **API Server:** Python-based FastAPI server that exposes core API endpoints to power all of Dabblebase's features, including student project management and Dabblebase's database, authentication, and storage features.
- **Task Queue:** Python-based task queue that schedules and completes expensive blocking tasks, including publishing assignments and spinning up databases.
- **Cron Scheduler:** Python-based process that schedules checks necessary for listening to realtime database changes.
- **Realtime Server:** Elixir-based server that manages Dabblebase's realtime functionality and manages clients connected via websockets.
- **PgBouncer:** Manages connections to the content database by students' project backends.

These components rely on the following data sources:

- **Main (admin) database:** PostgreSQL database that keeps track of active users, courses, assignments, and projects.
- **Content database cluster:** PostgreSQL database cluster that contains the individual databases for student projects.
- **Redis instance:** Redis key-value store that powers the task queue.
- **S3 instance:** S3-compatible object storage instance to power Dabblebase Storage.

One of the core design goals of Dabblebase is that project can be *self-hostable*. To make hosting easy regardless of the preferred environment, Dabblebase's core services are containerized in Docker containers. These containers can be self-hosted in a Kubernetes cluster available to the university department or instructor. In addition, instructors can host different pieces

of Dabblebase’s functionality in different places to fit their needs. The main FastAPI server and realtime server functionality may be hosted in a Kubernetes cluster in combination with external PostgreSQL, Redis, or S3 providers. Dabblebase can also be extended or modified to support additional authentication providers beyond UNC authentication for use in other institutions or contexts.

Database

Provider

Initially, SQLite emerged as a great potential provider option. Databases are stored as single `.sqlite` files, making it easy to organize and make accessible to students while also providing inherent isolation between databases. However, in practice, there are problems with SQLite:

- **Limited functionality:** SQLite supports a subset of SQL with looser type safety and less querying options. In addition, SQLite databases do not support functions and triggers, features which are often useful for certain web applications.
- **No realtime listening support:** Without support for database triggers and functions, as well as no support for notifications or logical replication, realtime functionality with SQLite would be extremely difficult to implement and scale to support hundreds of databases.
- **Less real-world applications:** While SQLite is a great database provider for single-user applications, mobile applications, or embedded into smaller applications, it lacks the tooling for larger applications. Most production web applications opt for a more robust database provider such as PostgreSQL or MySQL. Dabblebase’s core design principles stress the importance of implementing features in a way that is transferrable for students.

For these reasons, SQLite was not ideal for powering Dabblebase’s database features.

PostgreSQL is one of the most robust and widely adopted database providers. Developers regularly cite PostgreSQL as the most common database provider they work with in industry [2]. Postgres also is cited as the most SQL-standard compliant of the major database providers, and is open-source. Postgres also offers all of the features needed for realtime listening, including functions, triggers, and `pg_notify`.

Having access to their own personal PostgreSQL databases for projects provides great value to students across many courses, including courses focused on databases.

Multi-Tenant Structure

The core objective of Dabblebase’s database feature is to provide a private, isolated database for every student project. One of the design goals for Dabblebase was *minimal vendor lock-in*. Students should be able to connect to their database directly or by using any variety of database interfacing tools (such as ORMs). The ideal way to accomplish this is for Dabblebase to provide a direct database URL per student project, like so:

```
postgresql://user:password@host/db
```

With this URL, students would be able to connect to their database using any tool they wish. Nearly all major ORMs (particularly, SQLAlchemy for Python, Prisma and Drizzle for TypeScript) connect to databases via a database URL [3].

Creating individual database instances for every student project is not feasible, since it scales incredibly poorly. To provide a database URL to a private, isolated database for every student project, Dabblebase must use a **multi-tenant** architecture for student project databases, where each student project is considered a tenant of the database service. The idea is to use one single Postgres database instance that encapsulates all student project databases.

There are different approaches to setting up a multi-tenant architecture. These approaches are summarized below:

Approach	Shared DB, Separate Schemas In this approach, the database cluster contains one single database that all students connect to. Separate database schemas would be created for every project.	Separate Databases In this approach, the database cluster contains multiple databases, where each constituent database is associated with one student project.
Access Method	Database roles would be added for every student project, and these roles would be scoped to only have access to the project schema (by altering the role's <code>search_path</code> to be set to the schema). Database users assigned these roles would be made for each student. The students would then be able to connect to the database using the database URL: <code>postgresql://user:password@host/shared-db</code> When a student user connects to their database using this URL, they would see just the tables in their schema and have full access to modify only their schema.	Database users would be added to every student project database with full admin permissions over their allocated database. Then, students would be able to connect to their database by specifying their assigned database in the database URL: <code>postgresql://user:password@host/project-db</code> When a user connects to their database using this URL, they will have full access and control over their database.
Isolation	Limited While this approach prevents users from directly accessing or <i>modifying</i> other schemas, it does not adequately isolate student projects entirely. All project schemas are stored in one database, so accessing the single database's metadata would leak information about project schemas and the tables located in each schema. This database metadata is accessible via the <code>pg_catalog</code> schema, which is built-in to every Postgres database and is <i>also automatically always part of a user's search path</i> [4]. This means that even if a Postgres user's search path is altered, the <code>pg_catalog</code> will still be searchable. Since all students would connect to the same shared database to access their schemas, all students have access to the same <code>pg_catalog</code> schema. The schema is also plainly visible in many GUI-based database tools. Using this access, students may be able to discern details about the configuration of other students' databases, which may be exploited to cheat on assignments.	Great Since projects are assigned entirely separate databases, they are entirely isolated from one another. The problem with leaking data via <code>pg_catalog</code> as described in the other approach would not occur, since students accessing their database with their database URL would see just the metadata for their database and not other databases in the cluster. It is possible that students can see the <i>names</i> of other databases in the cluster using GUI-based database tools, but they would be able to learn nothing else about these other databases.

Features	Good with Limitations Most database features can be used by students with this approach, including creating / modifying / deleting tables abd data, creating Postgres functions and triggers, and more, as long as the role permissions are set correctly. There are some limitations, however. Since students are scoped to one schema, students cannot create or access multiple schemas if their project required it.	Great Since students have access to their own dedicated database for their projects, they would be able to use all Postgres database features, including creating and accessing other schemas.
Scalability	Great There is no upper limit to the number of schemas that a Postgres database can hold (other than the existing limit of the maximum number of tables in the database, which is over 4 billion), meaning that Dabblebase could scale to potentially millions of projects with just the single database [5].	Good with Limitations There is also no hard upper limit to the number of databases that a single Postgres instance can contain, and instances can typically scale well tens of thousands of databases. However, since the creation and management of multiple databases is more resource-intensive, this approach practically can scale to less projects than the multi-schema approach. This limit likely would not be hit if used by one class or even one department, but might if one database is used to support multiple semesters or multiple schools.

Dabblebase’s database feature implements its multi-tenant architecture using the **separate databases** approach. Since Dabblebase is designed to be used in an educational environment, strong project isolation was essential and ultimately the deciding factor.

This approach would likely not encounter scaling issues for a while when used for a single class or a small set of classes. In the event that Dabblebase is adopted more widely, these scaling issues could be remedied with *database sharding*. Database sharding is a horizontal scaling approach in which project databases can be distributed evenly across multiple database instances instead of just one. For example, a separate database instance could be initialized for each department adopting Dabblebase, and all student projects databases would be stored in the instance corresponding to the department a student is in, as shown in the figure below:

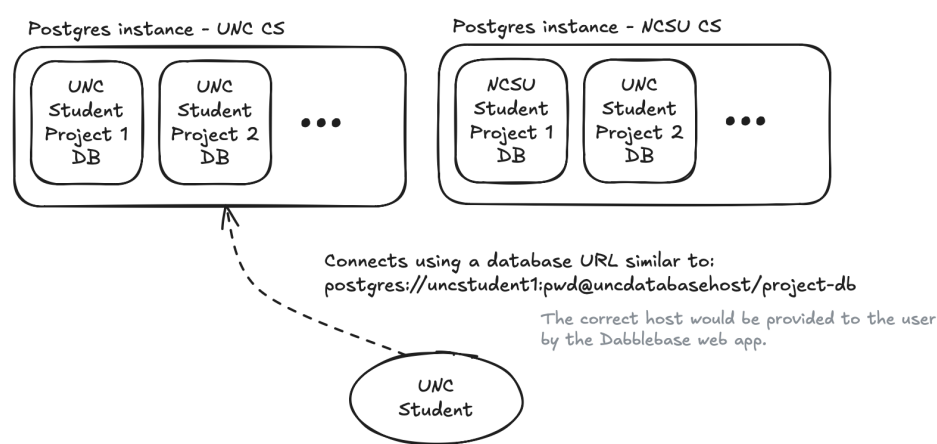


Figure 2: Example setup with database sharding

User Management

As described in the *Access Method* section in the "Separate Databases" column of the multi-tenant database approach table, a database user is added for every student project database. This user has full admin permissions for the database. Students are given the username and password as part of the database URL for their project. This approach works well for both individual and group projects. Instructors can also access student databases using these credentials.

Connection Pooling

Once provided the database URL, students can connect their projects to the database and perform operations on it using an ORM.

PostgreSQL implements a "process per user" client/server model, meaning Postgres creates a separate backend process for each client connection that serves the connection. While this works well for databases serving as the backend for a few apps, it scales very poorly to hundreds or more concurrent connections.

Consider a course like *COMP 426: Modern Web Programming*, which might have 200 students per semester. Assuming that each project has a running deployment that connects to a Dabblebase database, and a student has 3 projects deployed - the number of connections to Dabblebase's database server is approximately 600! Connecting to PostgreSQL directly would likely not even scale for a single class.

To solve this problem, a *connection pooler* called **PgBouncer** sits in front of the content database instance. PgBouncer maintains a pool of already-opened Postgres connections ready to use. Instead of student projects connecting directly to the Postgres database instance, student projects would instead connect to the PgBouncer instance. PgBouncer would then reuse its small set of database connections and assign them to different client requests as needed [6].

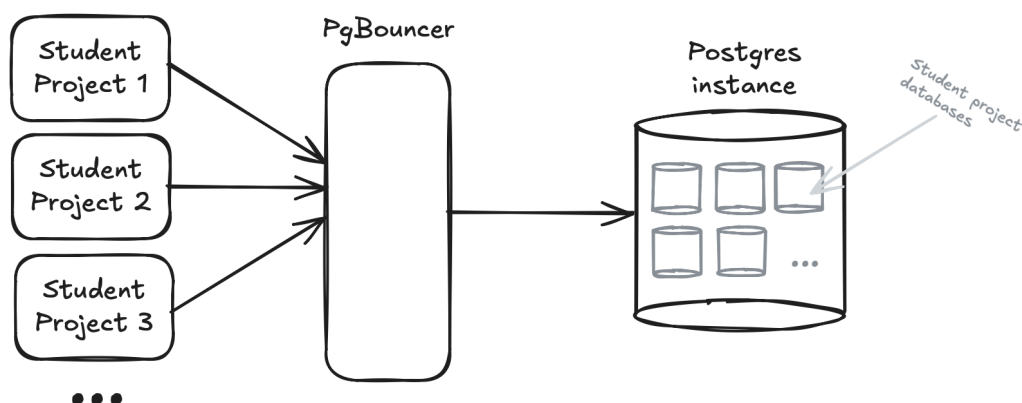


Figure 3: Connection pooling using PgBouncer

The PgBouncer instance would be exposed as its own host route and port, so students would connect to their project databases using a URL like so:

```
postgresql://user:password@pgbouncerhost:pgbouncerport/project-db
```

This URL would be provided to students via the Dabblebase web app interface.

With this configuration of PostgreSQL, a multi-database multi-tenant configuration, and connection pooling, Dabblebase's database feature is able to support the creation of isolated, private databases for student projects.

Authentication

Authentication Methods

There are two primary methods used by servers to authenticate incoming requests:

- **Session-based Authentication:** The server keeps track of *sessions* in a database. When a user logs in, a new session is created (typically with information about the user, the time the user logged in, and session expiry time). The server then sets a cookie in the user's browser that includes the session ID. On subsequent requests, the user will pass the session ID cookie along with the request. The server will query the database to verify that the session is valid, and if so, the server responds to the request. Otherwise, the request is rejected.
- **Token-based Authentication:** When a user logs in, the server will *issue a JWT* (*JSON Web Token*) and sets the JWT as a cookie in the user's browser. JWTs have three main components:
 - **Header:** Contains metadata about the token, including the algorithm used to sign the token.
 - **Payload:** Contains some data from the server, often about the authenticated user (such as a user ID, email, or name).
 - **Signature:** Contains a cryptographic signature used to verify that the token was not tampered with.

The server will first look up the user in the database to ensure that they exist, then creates the payload for the token. This might look something like `{"userId":1234}`. Then, the server will run the payload through a message authentication code algorithm that takes in a payload message (m) and a *secret key* (k_s) to generate a signature (s). This secret key is known only to the server and is stored in its environment. Commonly, HMAC-SHA256 (abbreviated below HMAC) as is used as the algorithm in JWT signing like so:

$$s \leftarrow \text{HMAC}_{k_s}(m)$$

This signature is then included as part of the JWT the server puts in the user's cookies.

On subsequent requests, the user will pass the JWT cookie along with the request. The server will then *verify* that the token is legitimate (and signed by itself, ensuring the user was properly authenticated). The token the server receives from the user's cookies should include a message (m') and a signature (s').

To verify the token, the server will run the hashing algorithm using the secret key on the new message m' . If the output of this operation is the signature s' , then the token must have been signed by the server using k_s . If the algorithm does not produce s' , that indicates the token was not signed by the server using k_s and the token might be fraudulent:

$$\text{Token is valid } (s' \text{ was signed using } k_s) \iff \text{HMAC}_{k_s}(m') = s'$$

For Dabblebase's authentication feature, *token-based authentication* is preferred over session-based authentication. With session-based authentication, Dabblebase's server would need to store the active sessions for all users of all student projects, and student projects' backends would need to query session information from Dabblebase on every request, which is slower and less scalable.

With token-based authentication, student projects' backends only need to *verify* that a JWT token was signed by the Dabblebase server - no database queries are necessary. This setup is shown in the figure below:

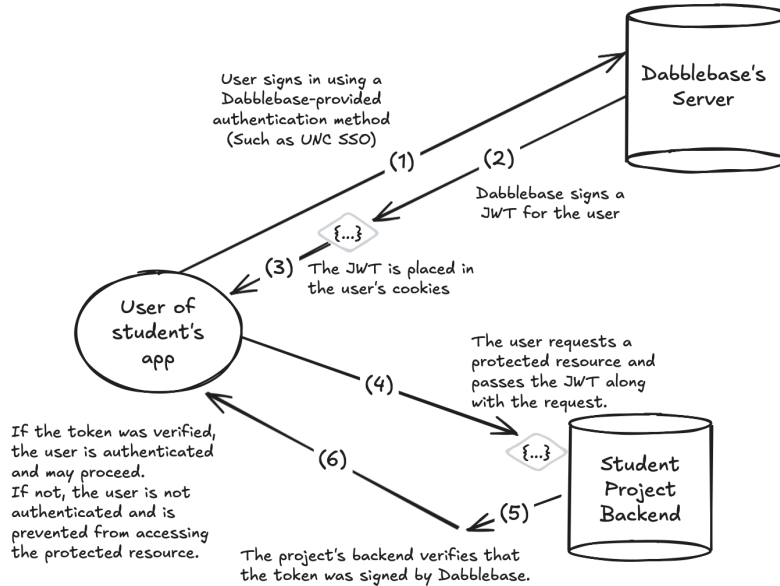


Figure 4: Token-based Authentication with Dabblebase

The only issue with this approach, however, is that for the student project backend to verify the JWT, the backend must have access to Dabblebase's secret key k_s . This secret key should not be made public, since its exposure can allow any malicious user to forge their own JWTs.

Authentication via Asymmetric JWT Signing

This is ultimately an issue because the hashing algorithm used in the example above, HMAC, is a *symmetric* message authentication code algorithm - meaning that the same key must be used to both sign and verify the token.

To solve this problem, instead of signing JWT tokens using a symmetric message authentication code algorithm, Dabblebase uses an asymmetric digital signature algorithm. The idea is that instead of using one secret key k_s for both the signing and verification steps, one key k_s should be used to sign the JWT (the signing key), and another key k_v should be used to verify the signature (the verification key). This class of algorithms are built on top of trapdoor permutations (TDPs) which contain three main components:

- A *signing algorithm* ($\text{SIGN}_{k_s}(m)$) that produces a signature s by signing a payload m using k_s .
- A *verifying algorithm* ($\text{VERIFY}_{k_v}(m, s)$) that returns `true` if the signature is valid.
- A *key generation algorithm* ($\text{KEYGEN}()$) that produces k_s and k_v such that $\text{VERIFY}_{k_v}(m, s)$ verifies s if and only if s was produced by $\text{SIGN}_{k_s}(m)$.

Dabblebase uses RSA as its asymmetric digital signature algorithm. RSA includes `RSA_SIGN`, `RSA_VERIFY`, and `RSA_KEYGEN` - the three components listed above.

When a new student project is created, Dabblebase uses the RSA key generation algorithm to generate a signing and verification key for the project:

$$k_{s_{\text{project}}}, k_{v_{\text{project}}} \leftarrow \text{RSA_KEYGEN}()$$

Dabblebase keeps the signing key $k_{s_{\text{project}}}$ private and uses $k_{s_{\text{project}}}$ to *sign* JWT tokens for authentication requests made for the project. Then, Dabblebase distributes the verification key $k_{v_{\text{project}}}$ to the student developers of the project to use in their project's backend to verify JWT tokens. With this approach, the private signing key is never publicly revealed. The revealed verification key can be used to verify JWT tokens but cannot be used to forge new ones. This setup is shown in the figure below:

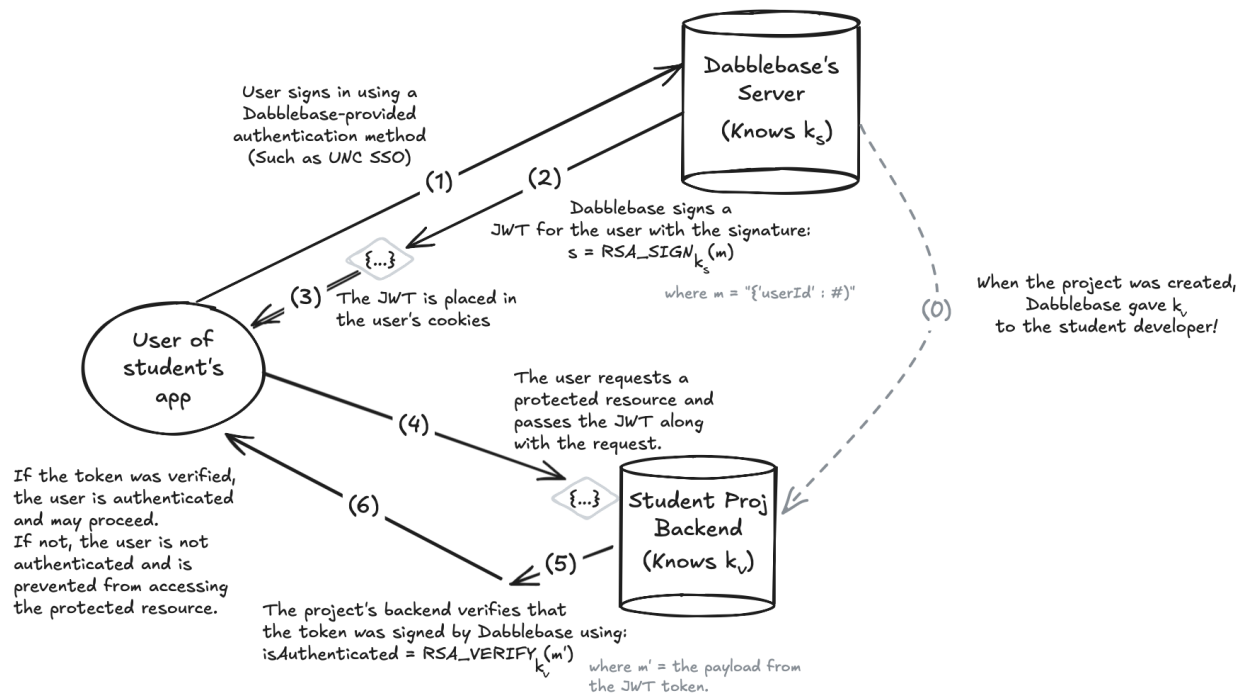


Figure 5: Asymmetric JWT Signing with RSA

As alluded to above, Dabblebase keeps track of all users who register with Dabblebase Authentication. The JWT token's payload includes the user's ID, which can be used by the student project backend to identify the authenticated user. The student developer can use their discretion to create their own `users` database table if they wish to store extra app-specific information about the user.

Authentication Providers

Typically, authentication services such as Supabase Auth, Clerk, and others allow users to sign up in different ways such as with an email / password combination, Google Sign-in, and more. Dabblebase allows users to sign up with a university's SSO system. Dabblebase routes authentication requests through UNC SSO, but more authentication providers for other universities or other services (such as Google) can easily be added.

Dabblebase Server

Dabblebase's core database and authentication features are served with Dabblebase's API service, a FastAPI-based server written in Python. In addition to the database cluster that stores the databases for student projects (the *content database cluster*), Dabblebase also connects to a second database cluster responsible for keeping track of all Dabblebase-related data such as courses, projects, and users (the *admin database cluster*). The server exposes the APIs that Dabblebase's web application interface with.

Some processes, especially managing assignments, can be expensive. For example, when an assignment is created, Dabblebase needs to create a database for every student (or group) in the course. To facilitate this, Dabblebase's API service relies on a task queue for expensive batch operations to perform asynchronously.

Realtime

The third core feature of Dabblebase is **Dabblebase Realtime** which exposes web socket endpoints for student projects to enable realtime updates.

Unlike the Dabblebase database, where each project is expected to use only a handful of connections (typically just a single persistent connection between the project's backend server and its database), web socket connections are established directly with clients in the browser. This means that a web socket connection would be established for every concurrent user of a student project, meaning that it is reasonable to assume Dabblebase Realtime could need to support thousands of concurrent connections at once.

Dabblebase’s Python and FastAPI-based server can expose web socket endpoints directly, however, scaling becomes very challenging since this involves orchestrating hundreds or thousands of connections in Python. Python itself is not designed to juggle such a large number of processes at once.

Realtime Server with Phoenix and Elixir

Instead, Dabblebase Realtime is powered by a separate, dedicated realtime server, written using the Phoenix framework for the Elixir programming language.

Elixir is the perfect language to power the realtime server. Elixir runs on the Erlang virtual machine (BEAM) and adopts the BEAM concurrency model. This concurrency model relies on actors, contained lightweight processes that can communicate with each other through message passing. BEAM can easily handle millions of these processes. The Phoenix framework uses the power of Elixir’s concurrency model to provide the infrastructure for serving web socket endpoints that scale easily for millions of concurrent connections [7].

When a client connects to a Phoenix server via a web socket, they connect to a *channel* and a certain topic within the channel. Clients can push messages to topics they have joined and receive messages from other clients who push messages to the topic.

Remember that Dabblebase Realtime encapsulates three main features:

- **Database Changes:** Student projects can listen for realtime changes to data in their database and respond to those changes in their application’s frontend.
- **Presence:** Students can configure their projects to be able to see who is currently logged in to their apps.
- **Broadcast:** Student projects can set up channels to send messages back and forth between clients in realtime.

To support these features, Dabblebase’s realtime server exposes one websocket endpoint. When the client connects to the websocket endpoint, it can choose from one of three channels to connect to, with each channel corresponding to a realtime feature. Channel names are typically formatted as `<channelname>:<topic>:<subtopic?>`. Dabblebase uses the topic slot to specify the project to connect to (by its ID), and optionally can accept a subtopic for listening to specific types of broadcasts for a project.

For example, a client seeking presence information for a student project with ID 1 might connect to the channel `presence:1`. The figure below shows the format for the topic names based on the type of realtime the client is connecting to:

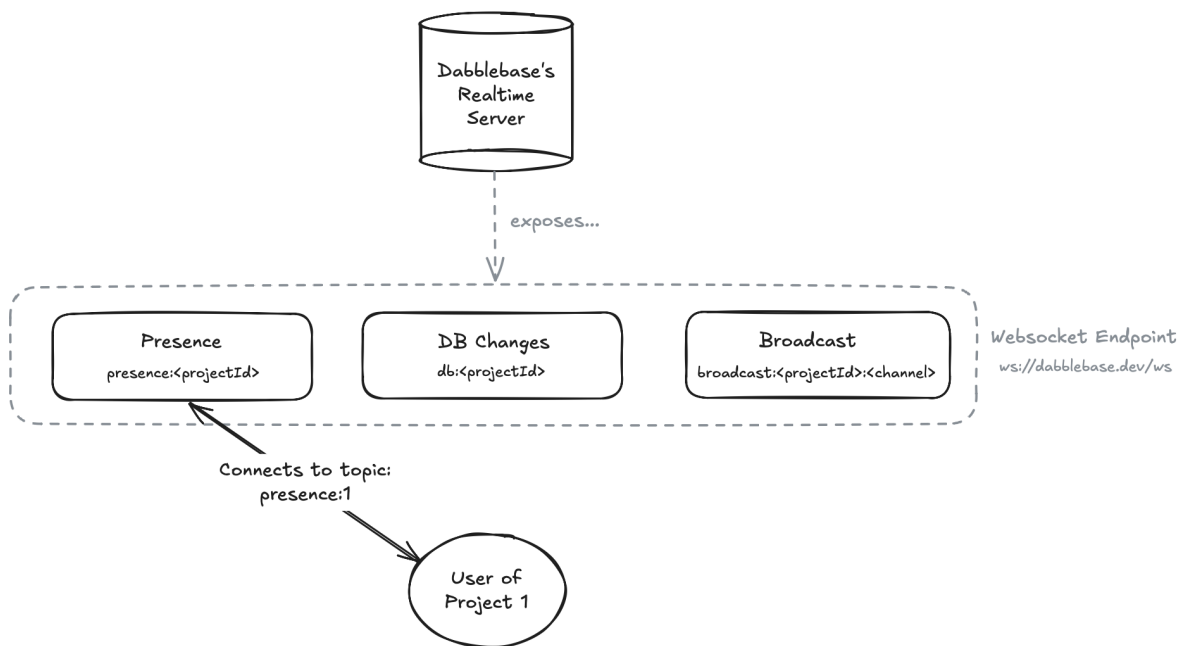


Figure 6: Realtime channels configuration

When connecting to the web socket, the client may also pass tokens, including its authentication token, to the realtime server if authentication is required.

Ultimately, clients must use Phoenix's JavaScript library to connect to the web socket. For example, this is the JavaScript code for a client to connect to the presence channel for a project with ID 1:

```
1 import { Socket, Channel } from "phoenix";
2
3 const socket = new Socket("ws://localhost:8000/ws", {
4   params: { auth_token: "..." },
5 });
6
7 socket.connect();
8 const channel = socket.channel("presence:1", {});
9
10 channel
11   .join()
12   .receive("ok", (_) => console.log("Joined channel"))
13   .receive("error", (_) => console.error("Failed to join"));
14
15 // Listen for a presence update
16 channel.on("presence_state", (state) => {
17   console.log(state);
18 });
```

For ease of use for students, this functionality is abstracted behind the more user-friendly Dabblebase Client Realtime API, described later in this paper.

The Broadcast and Presence features are powered by functionality built-in to Phoenix. Listening to database changes, however, required more configuration.

Listening to Database Changes

To listen for database changes, a realtime trigger must be added to every table in every student project database. Whenever a table is modified, the trigger fires and notifies the realtime server using Postgres's `pg_notify` utility. Whenever the realtime service is notified of a change in a project database, it passes along the change to all connected clients listening to database changes for the project.

Since realtime triggers must be added to every table in all student project databases, and tables may be added to a project database *at any time* by a student, Dabblebase has a scheduled cron job to check all project databases to determine if tables have been added. If a table has been added, then the realtime database trigger is added to the new table, ensuring that the realtime server can always catch changes made in any table across any student database.

The workflow for broadcasting database changes is shown in the figure below:

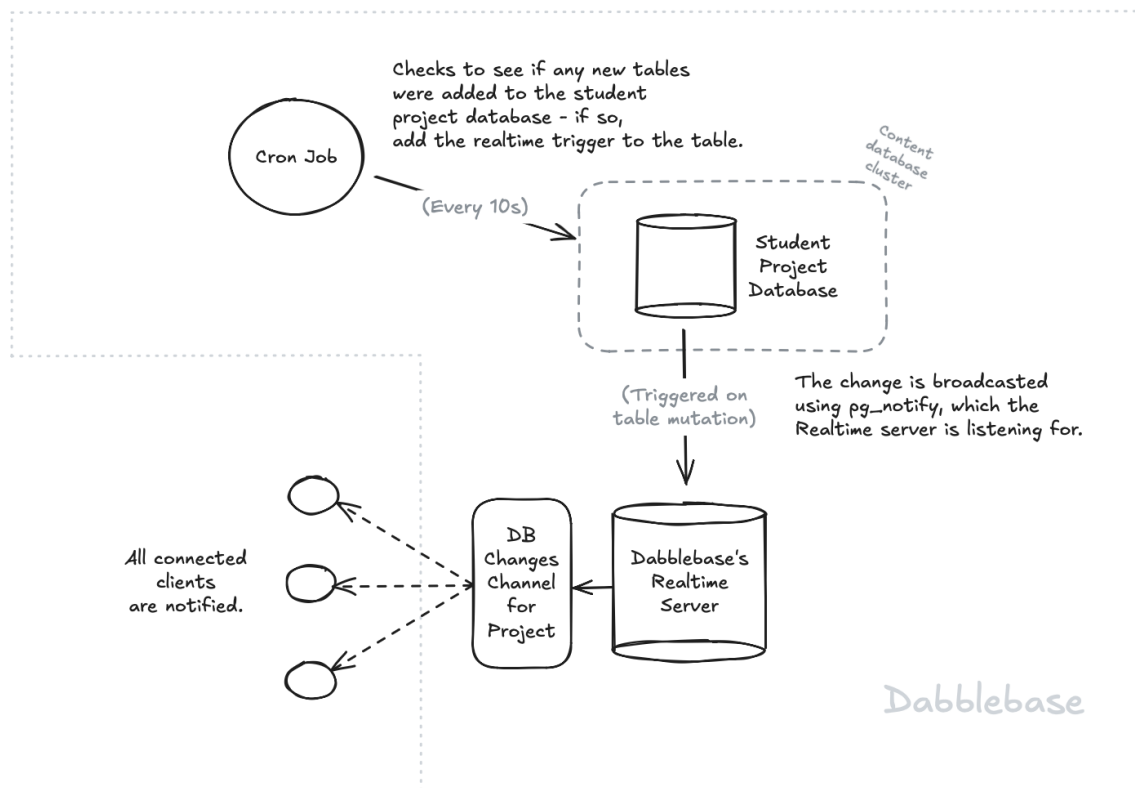


Figure 7: Realtime database changes

Storage

The final core feature of Dabblebase is **Dabblebase Storage**, which gives student developers access to cloud object storage for managing file uploads.

To power this object storage functionality, Dabblebase relies on an object storage instance compatible with AWS's S3 APIs. S3 is the industry standard solution for object storage. Dabblebase maintains one S3 bucket, with immediate subdirectories for each project by their ID. Student developers use Dabblebase APIs to get, upload, and download files related to their project.

To ensure that only authorized project code is able to call these APIs, Dabblebase's servers sign a *project token*, which is given to the student developer via the Dabblebase web interface. This token is passed along with every storage-related request for a project and authenticates the storage request. This same token is also used to authenticate project requests with Dabblebase's realtime web socket endpoints.

Web Application

Dabblebase's core assignment and project management features are accessible from the Dabblebase web application. This application is served from a web app server powered by TypeScript and Next.js.

Instructor View

On this site, instructors have the ability to manage courses and the course roster, create and publish new assignments, and access information about student projects after an assignment has been published.

The instructor's console view shows all of the assignments they have created, their publish status, and whether or not these assignments are group or individual projects:

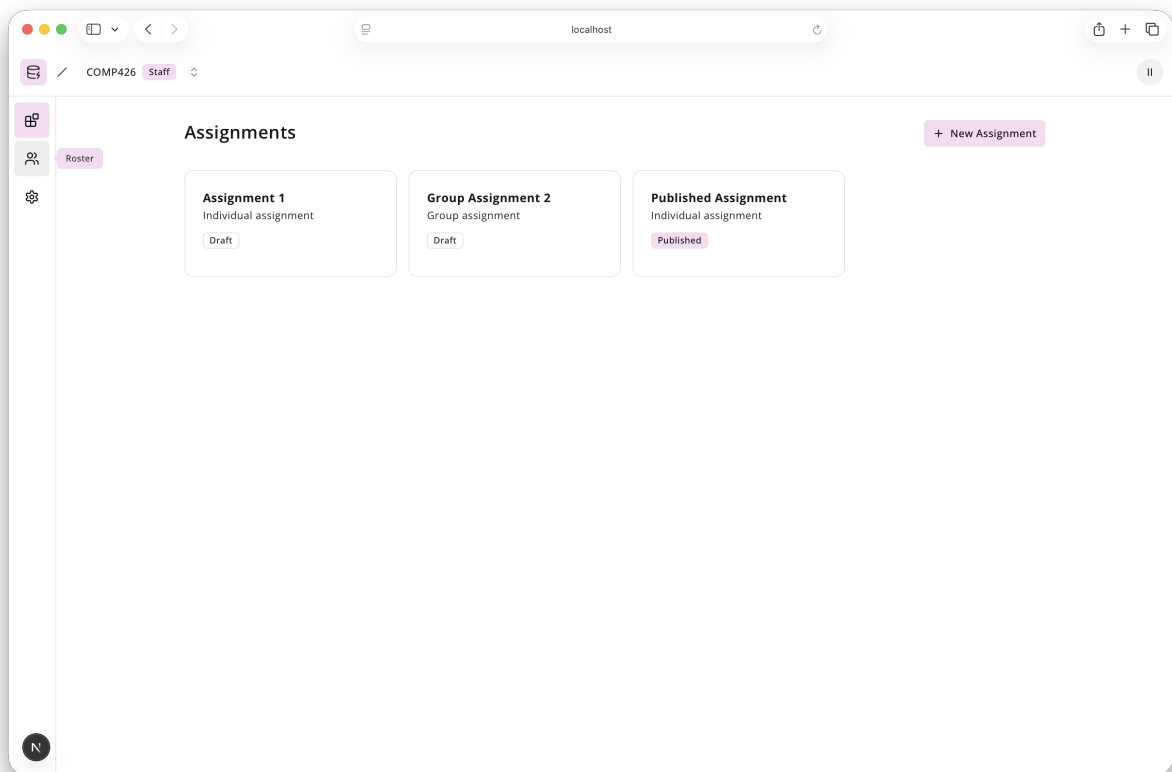


Figure 8: Instructor Portal

When creating a new assignment, instructors can select whether assignments are group or individual projects and provide a setup SQL script to run on all student databases when the assignment is published. Instructors can use this script to pre-populate tables or seed data in student databases, which may be useful for various types of assignments. Instructors can also configure groups based on their course’s roster:

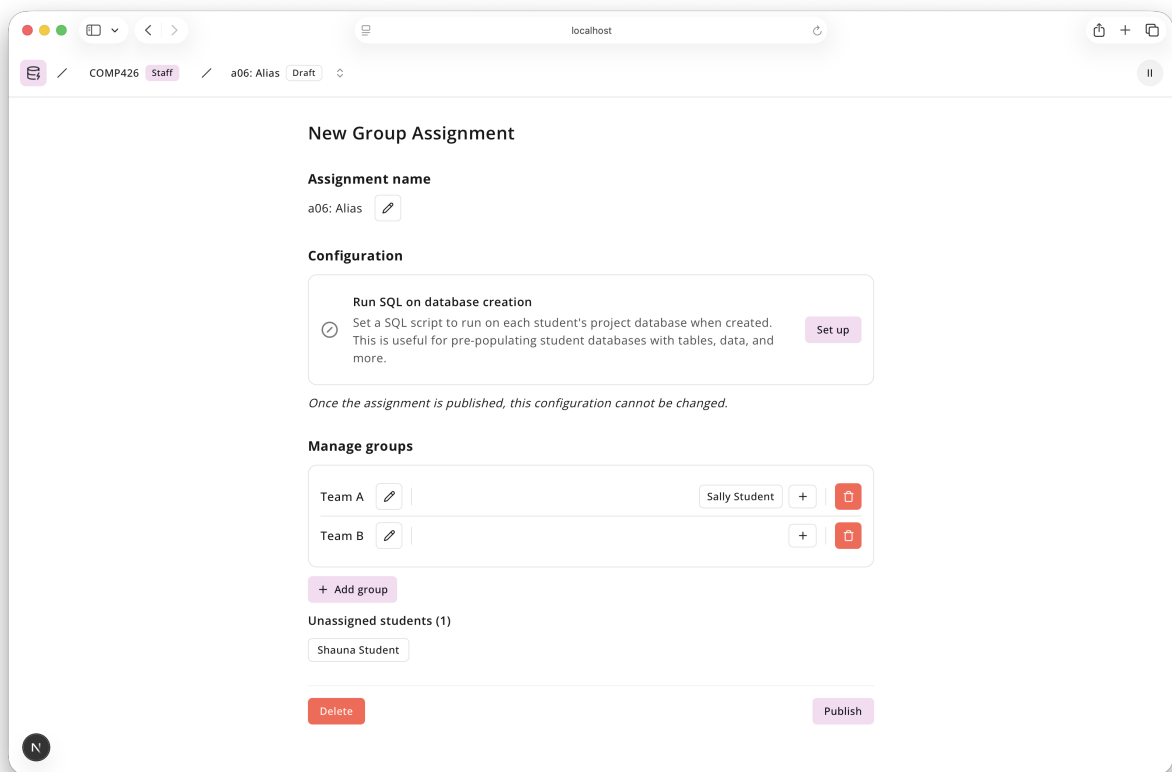


Figure 9: Instructor Draft Assignment View

After an assignment is published, instructors can view credentials to connect to student databases and even download the dump files for all databases, which is helpful for autograd-ing:

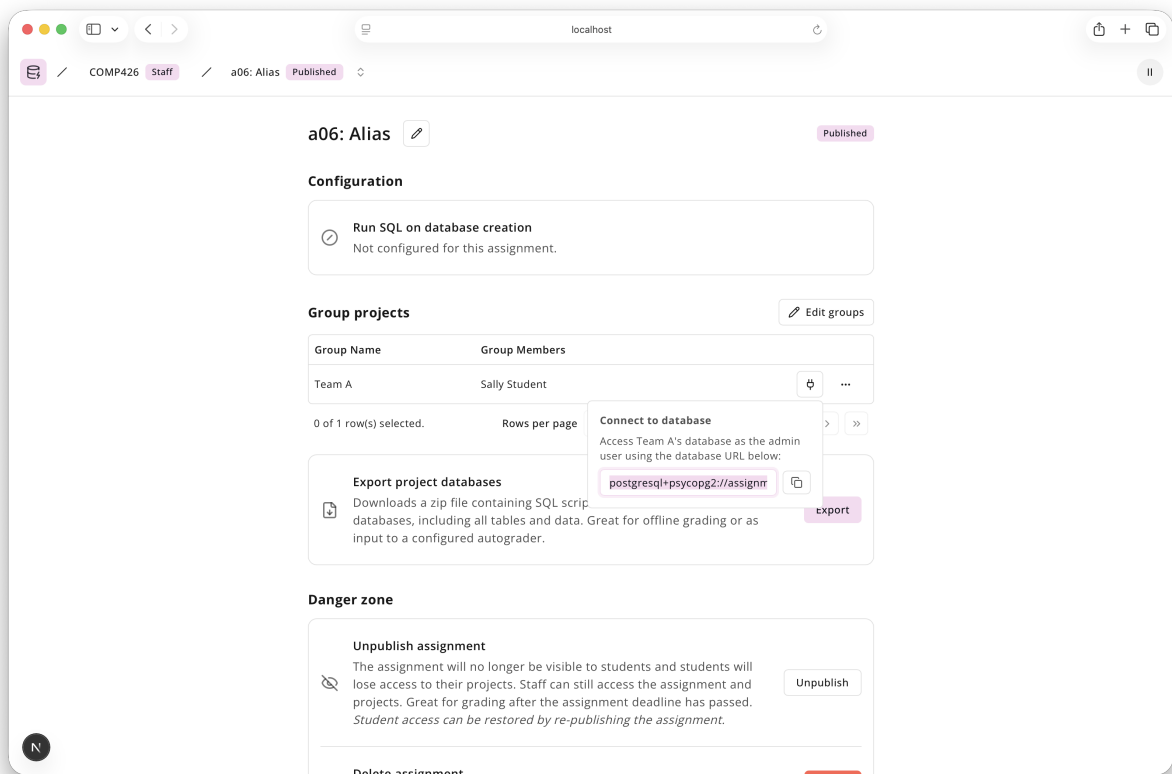


Figure 10: Instructor Published Assignment View

Student View

Students are able to see the courses they are a member of and all published assignments. When a student clicks on an assignment, they are given information about their project as well as important credentials required for them to access core Dabblebase features. These credentials include:

- **Database URL:** The direct URL to access their private PostgreSQL database provisioned by Dabblebase. Students can interact with their database by providing this database URL to any ORM of their choice.
- **Auth Verification Key:** The public verification key for the project generated by Dabblebase Auth. Students can use this key to verify that users' authentication tokens were legitimately signed by Dabblebase's Auth server for use with their project.
- **Project Token:** The project token, signed by Dabblebase, is used to authenticate the project with Dabblebase Storage and Dabblebase Realtime services.

These credentials are separated into different tabs like so:

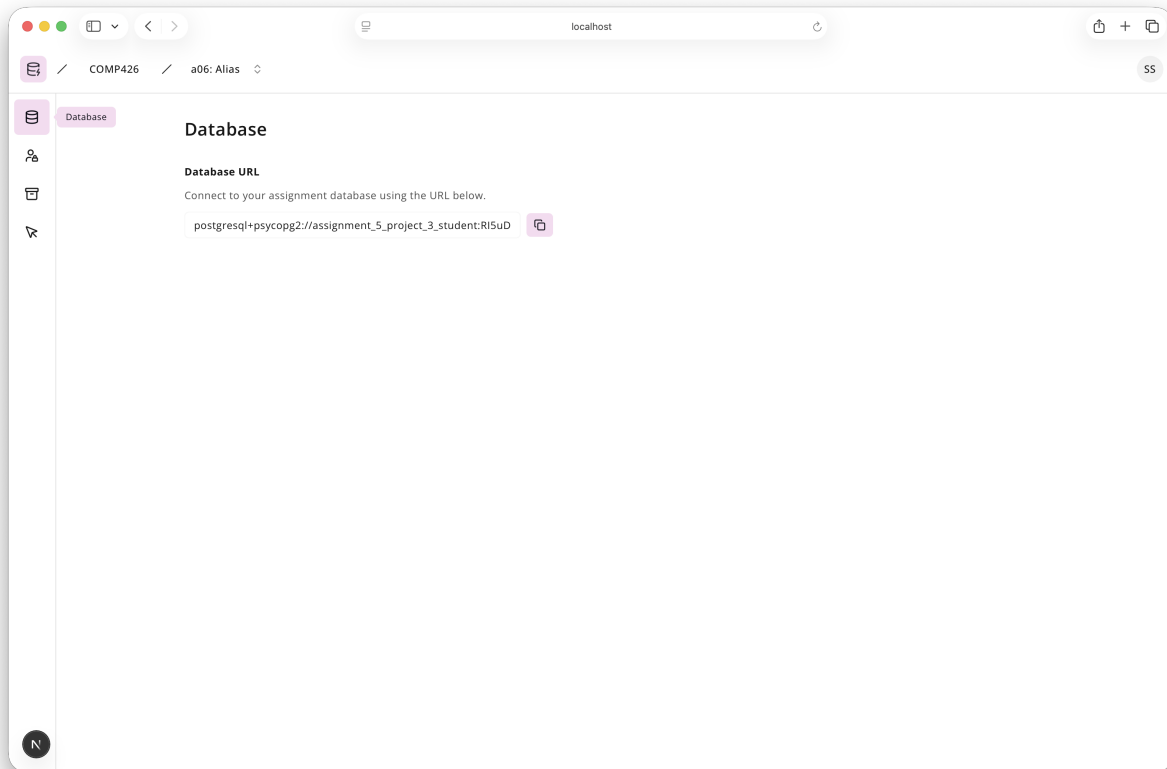


Figure 11: Student Portal

The website also provides information to the student on how to use these credentials in combination with the Dabblebase Client (see next section) to interact with Dabblebase services in their own applications.

Dabblebase’s Client Implementation

Dabblebase provides client libraries to enable students to interface with Dabblebase features for their project. In particular, the client library must allow students to do the following:

Dabblebase Auth

- **Authenticate:** Authenticate the current user and place the resulting auth token, signed by Dabblebase Auth, in the user’s cookies.
- **Sign out:** Remove the auth token from the user’s cookies.
- **Verify user:** Determine the authenticated user and verify that the user is authenticated legitimately by validating the auth token using the project’s auth verification key. This particular function would take in the auth token directly as a parameter, with the intention of surfacing the act of accessing the auth token from cookies to the student developer to implement.

Dabblebase Storage

- **Upload:** Upload files into the project’s object storage.
- **View:** View file(s) in the project’s object storage.
- **Delete:** Remove files in the project’s object storage.

Dabblebase Realtime

- **Connect:** Connect to any Dabblebase Realtime channel, listen to presence data, or database changes.

Note that the Dabblebase client library does *not* need to handle database concerns. Students would be recommended to use an ORM specific to their programming language to interact with their databases.

Client Availability and Design

To start with, Dabblebase provides a TypeScript client, `dabblebase-ts`, which performs all these functionalities natively and naturally to the programming language. The student developer passes in their credentials (auth verification key and project token) into the client constructor and exports the client to the rest of their TypeScript application. The client is fully documented and designed to be used both on the client-side and server-side, allowing students to build full-stack applications.

Dabblebase client functionality is extremely lightweight and can easily be replicated for other programming languages. Particularly, developing a Python client (`dabblebase-py`) or Java client is a natural next step due to its prevalence in many curricula as a server-side language. Dabblebase may even be used in mobile development courses if clients for languages like Swift (iOS) and Kotlin (Android) are developed.

The `dabblebase-ts` client is modeled similarly to Supabase's JavaScript API in usage. This ensures that Dabblebase's client follows established conventions and minimizes vendor lock-in.

Deploying Student Projects

Rather than deploying their projects with Dabblebase, instructors are free to let students deploy on any platform of their choice that is compatible the tech stack used in class. Like other BaaS providers, including Supabase, projects using Dabblebase can be deployed anywhere - both on servers directly (using a service like Carolina CloudApps or AWS) or on a serverless solution like Vercel. Platforms like Vercel and Netlify have greatly simplified deployment of full-stack web applications and also provide other services such as terrific CI/CD integration with GitHub and analytics tracking, features that provide significant pedagogical value in software engineering courses.

Achieving Design Goals

After discussing the implementation of Dabblebase's core features, we can evaluate how effectively Dabblebase achieves its design goals and principles.

Feature-Rich Tooling

Tools designed for pedagogical use often feel limited in capability. Dabblebase should offer the same core functionality as other BaaS services (like Supabase) to support a large variety of student projects.

Dabblebase offers the same the core feature set as premier BaaS providers like Supabase, including *authentication*, access to a private *PostgreSQL* database, *S3-like object storage*, and *realtime updates*. With Dabblebase, students are able to build the same complex projects they could build with Supabase, including social media and chatroom apps.

Minimal Vendor Lock-In

If students wish to further develop a project of theirs after a course finishes, they should be able to without having to rewrite large pieces of their app. Dabblebase should be designed in a way that makes it easy to switch to another infrastructure solution.

Dabblebase is designed to prevent vendor lock-in. For Dabblebase's database feature, students are simply given a database URL to connect their projects to and can use any ORM of their choice - Dabblebase does not require use of proprietary client APIs to interact with the database. If a student wishes to change their database provider in the future, they simply have to change the database URL their ORM is connected to and perform a database migration.

Dabblebase's authentication and storage features are accessed using a very minimal client API. To change providers, students would simply have to swap Dabblebase client functions in their codebase with functions for another client. Since Dabblebase Storage is implemented on top of an S3-compatible object store, the student would have to change very little about

the logic they use to manage file storage in their application if they switched to AWS S3 or a similar object storage provider.

Since Dabblebase’s realtime client API is designed similarly to Supabase’s realtime client API, if the student developer wishes to switch from Dabblebase to Supabase, little logic would change in the student’s codebase.

Flexible Support

Dabblebase should be flexible enough to work with a variety of tech stacks and tools to support the pedagogical goals for a variety of different courses.

Since Dabblebase exposes a direct database URL to students rather than a data management API, students may access their database in whichever way they please, supporting a large variety of contexts. Dabblebase can be used in a full-stack software engineering class, where students use an ORM to interact with their databases. Dabblebase can also be used in a database class, where students can view their databases in a GUI application or directly run SQL queries on their database.

In addition, with planned support for client libraries in many programming languages, Dabblebase can be used in classes using a variety of tech stacks.

Features for Instructors

Dabblebase should make it easy for instructors to create assignments with a customizable configuration to support a wide variety of assignments and courses. It should also be easy for instructional staff to access student projects and their databases for easy grading and debugging during office hours.

Dabblebase contains many features that make it easy for instructors to administer assignments. Instructors can create both individual or group projects, configure SQL scripts to run on every database when an assignment is published, and view the database URLs for each project for direct access into student databases. Instructors can also download dump files from each project database into a zip file, great for configuring autograders based on student databases.

Ease of Use

Since the core users of Dabblebase will be students who will often be implementing backend features for the first time, Dabblebase and its APIs must be written in a way that is easy to use, and its documentation must be presented in a way that is digestible and accessible.

Dabblebase’s web application contains easy-to-follow instructions and documentation that make using Dabblebase and its client libraries easy to use. Client libraries are also designed to fit well with programming language expectations and only expose necessary configuration so a significant amount of additional manual logic is not required to successfully use core Dabblebase features.

Pedagogical Value

Many existing BaaS solutions design its APIs in a way that obscures how features they provide work. Without compromising on usability, Dabblebase’s APIs should be designed in a way that reveals to students the core technologies behind how features work (such as cookies / JWTs for authentication, SQL for databases, or web sockets for realtime).

Dabblebase’s core features and client library are designed to maximize pedagogical value where important and not obscure important technologies behind how these features work. Many BaaS providers provide a proprietary way to access data from their provided database, obscuring the SQL operations required to query from the database. Since Dabblebase simply provides the student with a database URL, students use SQL or an ORM to interact with their database, allowing students to get familiarity with SQL databases and querying conventions.

Many BaaS client libraries, including Supabase, obscure the use of JWTs for authentication with abstract functions such as `getUser` that implicitly decode these tokens. With Supabase Auth, a developer never actually sees how authentication is working under the hood in their project.

Dabblebase's auth client library does not obscure the use of JWTs for authentication, since client functions for verifying the authenticated user require the student developer to supply an auth token as input. For example, in a server-side API route, the developer might verify a user like so:

```
1 // Server-side route (API)
2 const { subject, error } = dabblebase.auth.verify(
3   request.cookies["auth-token"]
4 );
```

With this approach, students develop a better sense of how authentication is implemented in their projects.

Self-Hostable and Open Source

Dabblebase should be able to be self-hosted by instructors or university departments so their students are able to use it. Dabblebase is a tool for educators and students alike, so it should be entirely open-source for future development.

As noted before, Dabblebase is an open-source project and is entirely self-hostable, allowing for deployment and use in various contexts.

Future Goals

While the current implementation of Dabblebase covers the intended core design goals, there are many features that would further improve Dabblebase:

- **More Authentication Providers:** It would be ideal to have a way for instructors to customize the authentication providers offered to students in their class. Particularly, adding support for customizable OAuth providers, as well as Google and GitHub sign-in methods, would be great for use cases outside of UNC.
- **GUI Database Manager:** Adding a database viewer built-in to the web application would make grading far more convenient for course staff and would also make the development workflow for students easier.
- **Expanded Client Library Coverage:** As mentioned earlier, adding client libraries for more programming languages including Python, Java, Swift, and Kotlin would allow for Dabblebase to be used by students in a larger variety of courses.
- **Advanced Features:** Other BaaS providers, including Supabase, support even more advanced features such as vector databases and hosting MCP servers for AI and ML-powered applications. There would be room to expand Dabblebase to support these features as well, or any other AI-related primitive that becomes mainstream in the future.

Conclusion

In this paper, I introduced Dabblebase, a Backend-as-a-Service platform designed for use by instructors and students in software engineering courses that does not compromise on features provided by other platforms. With Dabblebase, it becomes far more feasible for instructors to create and scale more ambitious software engineering courses with assignments that require a more complex infrastructure setup.

Dabblebase and its client APIs are all publicly available on GitHub here:

<https://github.com/dabblebase>

References

- [1] Deniz Akdur. “Analysis of Software Engineering Skills Gap in the Industry”. In: *ACM Trans. Comput. Educ.* 23.1 (Dec. 2022). URL: <https://doi.org/10.1145/3567837>.
- [2] Stack Overflow. *Stack Overflow Developer Survey 2025: Databases*. 2025. URL: <https://survey.stackoverflow.co/2025/technology/#2-databases> (visited on 10/24/2025).
- [3] Drizzle Team. *Getting Started with Drizzle ORM for PostgreSQL*. 2025. URL: <https://orm.drizzle.team/docs/get-started-postgresql> (visited on 10/24/2025).
- [4] PostgreSQL Global Development Group. *PostgreSQL Documentation: Schemas and the System Catalog*. 2025. URL: <https://www.postgresql.org/docs/current/ddl-schemas.html#DDL-SCHEMAS-CATALOG> (visited on 10/24/2025).
- [5] PostgreSQL Global Development Group. *PostgreSQL Documentation: Database Limits*. 2025. URL: <https://www.postgresql.org/docs/current/limits.html> (visited on 10/24/2025).
- [6] PgBouncer Project. *PgBouncer Features*. 2025. URL: <https://www.pgbouncer.org/features.html> (visited on 10/24/2025).
- [7] Phoenix Framework. *Phoenix Channels Guide*. 2025. URL: <https://hexdocs.pm/phoenix/channels.html> (visited on 10/24/2025).